

Pipeline Bill of Materials

A Zero-Trust Framework for Software Supply Chain Risk Visibility

Amina Emenena

George Washington University

ASBBS Conference 2026

The Blind Spot in Your Supply Chain

700%

increase in supply chain attacks since 2019

\$4.45M

average cost per breach

Sources: Sonatype 2022, IBM Security 2023

SBOMs: Necessary but Insufficient

Software Bills of Materials track code dependencies...

But they're **blind** to:

- Build environment configurations
- Tool versions running in CI/CD
- Secret access patterns
- Runner integrity

"SBOMs document the ingredients, but not the kitchen."

Real-World Exploits

SOLARWINDS (2020)

Attackers compromised the *build system*, not the source code. Malware injected during compilation.

CODECOV (2021)

Tampered CI script exfiltrated secrets from thousands of organizations for months.

Common thread: Nobody was watching the pipeline.

Introducing PBOM

Pipeline Bill of Materials

Zero-trust visibility into your build environment

Zero Trust for CI/CD

Applying NIST 800-207 principles to build pipelines:

- **Never trust implicitly** - Every tool, secret, and configuration must be verified
- **Continuous verification** - Not just at deployment, but every build
- **Least privilege** - Minimal secret scope, restricted token permissions
- **Assume breach** - Design for detection, not just prevention

The Four-Axis Health Model

1. TOOL CURRENCY

Are Actions pinned to SHA digests or floating on mutable tags?

2. SECRET HYGIENE

Minimal scope? Leaking to logs? Token permissions?

3. BUILD PROVENANCE

Can artifacts trace back to commit, runner, workflow?

4. VULNERABILITY POSTURE

Dependabot alerts addressed? Dependencies current?

Conceptual Framework

Zero-Trust Principles (NIST 800-207)



Tool Currency

Secret Hygiene

Build Provenance

Vulnerability Posture



Composite Health Score (A-F)



Risk Communication

Compliance

Prevention

Research Propositions

- **P1:** PBOM achieves greater supply chain visibility than SBOM-only approaches
- **P2:** Zero-touch architectures achieve higher coverage than per-repo tools
- **P3:** Health scores enable more effective board-level communication
- **P4:** Continuous monitoring reveals risk undetected by traditional compliance

BuildGuard: The Implementation

Zero-touch architecture - No workflow changes required

- Hooks into GitHub org-level APIs
- Scans **1,000+ repos** without PRs or opt-in
- Produces per-repo health scores (A-F)
- Generates compliance-ready reports

```
$ buildguard scan --org mycompany  
Scanning 30 repositories...  
Overall Health: B (73/100)
```

Pilot Results

30 repositories | 60 days

Coverage & Performance

100%

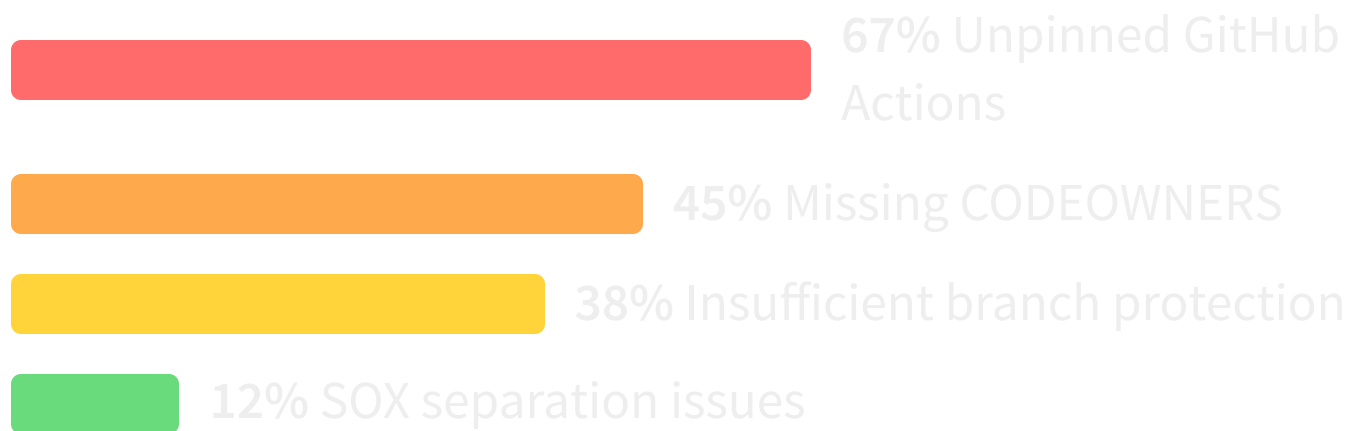
repository coverage

3.2s

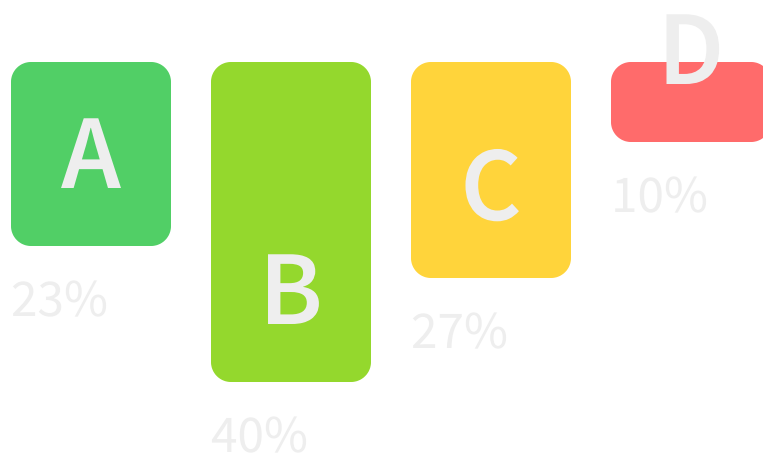
average time per repo

Zero developer friction - No workflow modifications needed

Most Common Violations



Health Score Distribution



Zero false positives in secret exposure detection

Value for Leadership

- **Quantify Exposure** - Translate pipeline drift into a single "Health Score" for board reporting
- **Scale Governance** - Cover 1,000+ repos without increasing headcount
- **Ensure Compliance** - Immutable evidence for audits and cyber insurance
- **Reduce Mean Time to Remediate** - Actionable insights, not alert fatigue

Key Takeaways

1. **SBOMs are not enough** - Build environments need their own bill of materials
2. **Zero-touch is possible** - Security visibility without developer friction
3. **Health scoring works** - Transforms raw metadata into actionable risk grades

Future Research

- Validation across diverse industry sectors
- GitLab and Azure DevOps support
- ML-enhanced detection for emerging threats
- Policy-as-code enforcement
- Integration with existing SBOM tooling

Thank You

Amina Emenena

amina@buildflowlabs.com

buildflowlabs.com

BuildGuard is open source

Speaker notes